Sequence Diagrams in Low-Level Design (LLD)

1. Introduction to Sequence Diagrams

A sequence diagram is a UML behavioral diagram that represents how different objects or components interact with one another to complete a particular operation.

It focuses on the **order in which messages are exchanged between participants** during the execution of a use case.

Example: Student Course Enrollment

Consider a college application where a student named Rahul wants to enroll in a Java Programming course.

From the student's perspective, the process involves:

1. Opening the college portal.
2. Selecting the Java Programming course.
3. Clicking the Enroll button.
4. Receiving an enrollment confirmation.

However, several operations must happen internally before the application can confirm enrollment.

The system may need to verify:

- Whether the student exists and has an active account.
- Whether the selected course exists.
- Whether the student is already enrolled.
- Whether seats are available in the course.
- Whether the enrollment can be saved successfully.

Only after these checks have passed can the system confirm the enrollment.

Why Class Diagrams Are Not Enough

A class diagram describes the static structure of a system.

For example, it can show that:

- A `Student` can have multiple `Enrollment` records.
- A `Course` can have multiple `Enrollment` records.
- Each `Enrollment` connects a student with a course.
- `EnrollmentService` contains the logic responsible for enrollment.

However, a class diagram does not explain the sequence in which these objects interact.

It cannot fully describe which component receives the initial request, which method executes next, or how the application handles an unsuccessful enrollment.

Sequence diagrams solve this problem by representing the interactions between objects in chronological order.

They are particularly useful in LLD for understanding method calls, service coordination, conditional execution, and the overall flow of a use case.

---

2. Basic Components of a Sequence Diagram

A sequence diagram contains several elements that represent participants, messages, and execution behavior.

2.1 Actors and Participants

A participant is an entity involved in an interaction.

Participants can represent users, objects, services, external systems, or other components.

An **actor** represents a role outside the system boundary that interacts with the system. It is commonly represented using a stick figure.

In the student enrollment example, the participants might include:

Participant	Responsibility
`Student`	Initiates the enrollment request
`CollegePortal`	Provides the interface for enrollment
`EnrollmentController`	Receives and handles the request
`EnrollmentService`	Executes the enrollment logic
`StudentRepository`	Retrieves student information
`CourseRepository`	Retrieves course information
`EnrollmentRepository`	Retrieves and saves enrollment records

Here, `Student` acts as an external actor, while the remaining participants represent application components.

2.2 Lifelines

A lifeline is the vertical dashed line extending downward from a participant.

It represents the participant's presence during an interaction.

For example, `EnrollmentService` has a lifeline because it participates in processing the enrollment request.

The vertical direction represents the progression of the interaction, with earlier events appearing above later events.

Important points:

- Each participant is represented by a separate lifeline.
- Messages are exchanged between lifelines.
- The interaction progresses from top to bottom.
- A lifeline does not represent a network connection or database connection.

The lifeline simply shows the participant's involvement over the course of the interaction.

2.3 Messages

Messages represent communication between participants.

In object-oriented applications, messages frequently correspond to method calls or their responses.

Consider the following Java statement:

```
enrollmentService.enroll(studentId, courseId);
```

Here, one object invokes the `enroll()` method on another object.

A sequence diagram represents this interaction using an arrow from the caller to the receiver.

Messages can represent method invocations, responses, signals, or other interactions relevant to the use case.

The type of arrow indicates the nature of the communication.

---

3. Types of Messages

3.1 Synchronous Messages

A synchronous message represents an interaction in which the sender waits for the invoked operation to complete before proceeding.

Consider:

```
Enrollment enrollment =
    enrollmentService.enroll(studentId, courseId);
```

The controller invokes the `enroll()` method of `EnrollmentService`.

In a normal synchronous Java method call:

1. The controller invokes the method.
2. Control enters the service method.
3. The service executes its logic.
4. The service returns a result or throws an exception.
5. The controller resumes execution.

The controller cannot execute the statement following the method call until that call completes.

UML notation: A solid line with a filled arrowhead represents a synchronous call.

Example:

`EnrollmentController → EnrollmentService: enroll(studentId, courseId)`

Synchronous messages are commonly used for regular Java method calls, validations, repository operations, and interactions where the caller waits for a response.

3.2 Asynchronous Messages

An asynchronous message represents communication in which the sender does not have to wait for the receiver to finish processing the message.

Consider the student enrollment application.

After successfully enrolling a student, the system may need to send an email confirmation.

The enrollment operation does not necessarily need to wait until the email has been delivered.

Instead, the application can publish an event to a message broker such as Kafka or RabbitMQ.

A separate notification component can process that event.

For example:

```
notificationPublisher.publishEnrollmentEvent(enrollment);
```

The interaction might look like:

1. `EnrollmentService` successfully saves the enrollment.
2. `EnrollmentService` publishes an enrollment event.
3. A notification component receives the event.
4. The notification component processes the event and sends an email.

The sender can continue its processing without waiting for the entire notification operation to finish.

UML notation: A solid line with an open arrowhead represents an asynchronous message.

Important: Asynchronous communication does not automatically mean a new thread is created or that execution becomes faster. It means the sender does not have to wait for the receiver to complete the interaction. The actual implementation depends on the architecture being used.

3.3 Return Messages

A return message represents a response sent back to the participant that initiated an operation.

Consider a repository lookup:

```
Student student =
    studentRepository.findById(studentId)
        .orElseThrow();
```

The service requests student information from the repository.

The repository retrieves the relevant record and returns the result.

The interaction can be described as:

`EnrollmentService → StudentRepository: findById(studentId)`

`StudentRepository --→ EnrollmentService: Student`

UML notation: A dashed line with an open arrowhead represents a reply or return message.

The returned value can also be written beside the arrow.

Return messages are useful when the returned information affects subsequent operations.

They do not need to be shown for every method call, particularly when the response is obvious or unimportant to the diagram.

3.4 Comparison of Message Types

Message type	Notation	Meaning
Synchronous	Solid line, filled arrowhead	Sender waits for the operation to complete
Asynchronous	Solid line, open arrowhead	Sender need not wait for completion
Return	Dashed line, open arrowhead	Represents a response or returned value

4. Activation Bars

An activation bar is a narrow vertical rectangle drawn over a lifeline.

In UML, it is formally known as an **execution specification**.

It represents a period during which a participant is executing behavior.

Consider the following example:

```
public Enrollment enroll(
        long studentId,
        long courseId) {

    Student student =
        studentRepository.findById(studentId)
            .orElseThrow();

    // Additional enrollment logic

    return enrollment;
}
```

When `EnrollmentService` executes the `enroll()` method, its activation bar represents that execution.

During processing, the service calls `StudentRepository`.

The repository receives its own activation bar while it performs the requested operation.

After the repository returns the result, execution continues in the service.

Important Points

- Activation bars indicate execution behavior on a participant's lifeline.

- Nested method calls can be represented using additional activation bars.
- Activation bars help identify which component is currently responsible for processing.
- They are not exact measurements of processing time or CPU usage.

For most LLD interviews, understanding what activation bars represent is more important than drawing every activation with perfect precision.

---

5. Self-Calls

A self-call occurs when an object invokes another method belonging to the same object.

Consider:

```
public class EnrollmentService {

    public void enroll(
            long studentId,
            long courseId) {

        validateEnrollment(studentId, courseId);
    }

    private void validateEnrollment(
            long studentId,
            long courseId) {

        // Validation logic
    }
}
```

The `enroll()` method calls `validateEnrollment()` within the same `EnrollmentService` object.

No other object is involved in this method invocation.

Representation in Sequence Diagrams

A self-call is shown using a message arrow that starts and ends on the same lifeline.

The arrow typically leaves the lifeline, bends to the right, and returns to the same lifeline.

A nested activation bar may be used to represent the execution of the called method.

Self-calls are useful for illustrating internal validation, calculations, helper methods, and recursive operations.

---

6. Object Creation and Destruction

Sequence diagrams can also represent objects that are created or destroyed during an interaction.

6.1 Object Creation

Consider:

```
Enrollment enrollment = new Enrollment(
    student,
    course,
    LocalDateTime.now()
);
```

A new `Enrollment` object is created during the execution of the enrollment operation.

In a sequence diagram, object creation is represented by a creation message directed toward the newly created object.

The new object's lifeline begins at its creation point rather than at the top of the diagram.

This helps distinguish objects that already existed from objects created as part of the current operation.

6.2 Object Destruction

UML also supports representing the destruction of an object.

Object destruction is shown using an `X` at the end of its lifeline.

For example, a system might create a temporary session object and later terminate that object's lifetime.

Object creation and destruction are generally included when object lifecycle management is relevant to the design.

They do not need to be shown for every object involved in an application.

---

7. Combined Fragments

Not every interaction follows a single, fixed sequence of operations.

Some operations are conditional. Others execute repeatedly.

UML provides **combined fragments** to represent these behaviors within sequence diagrams.

A combined fragment is shown using a rectangular frame enclosing the relevant interactions.

The fragment type is written in the upper-left corner.

Common fragments include:

- `alt` — Alternative execution paths
- `opt` — Optional execution
- `loop` — Repeated execution

7.1 Alternative Execution: `alt`

The `alt` fragment represents alternative paths that depend on conditions.

It is similar to an `if / else if / else` structure.

Consider the following enrollment validations:

```
if (!student.isActive()) {
    throw new IllegalStateException(
        "Student account is inactive"
    );
}

if (alreadyEnrolled) {
    throw new IllegalStateException(
        "Already enrolled"
```

```
    );
}

if (enrolledCount >= course.getCapacity()) {
    throw new IllegalStateException(
        "Course is full"
    );
}

enrollmentRepository.save(enrollment);
```

The result of the enrollment request depends on these checks.

Possible outcomes include:

Condition	System behavior
Student account is inactive	Reject enrollment
Student is already enrolled	Reject duplicate enrollment
Course has no available seats	Reject enrollment
All validations pass	Save enrollment

An `alt` fragment can represent these alternative outcomes.

Each alternative is placed inside a separate operand, with a guard condition written in square brackets.

For example:

```
alt
    [Student inactive]
        Return enrollment error

    [Already enrolled]
        Return duplicate enrollment error

    [Course full]
        Return capacity error

    [else]
        Save enrollment
        Return confirmation
```

In a UML diagram, horizontal dashed separators divide the alternatives.

Only the applicable execution path is followed for a particular request.

When to use `alt`:

Use it when different conditions lead to different interactions, such as successful and failed payments, valid and invalid login attempts, or successful and unsuccessful course enrollments.

Note: For sequential validation checks, an actual diagram may use multiple fragments or model the decision flow at an appropriate level of detail. The goal is to make the alternative behavior clear without misrepresenting the execution order.

7.2 Optional Execution: `opt`

The `opt` fragment represents an interaction that executes only when a particular condition is true.

Unlike `alt`, there is no separate alternative behavior required when the condition is false.

Consider a student who has enabled email notifications.

```
if (student.wantsEmailNotifications()) {
    notificationPublisher.publishEnrollmentEvent(
        enrollment
    );
}
```

The enrollment should succeed regardless of whether the student wants email notifications.

However, the notification event is published only when email notifications are enabled.

The sequence diagram can represent this using:

```
opt [Email notifications enabled]

    EnrollmentService
        → NotificationPublisher:
          publishEnrollmentEvent(enrollment)
```

If the condition evaluates to false, the enclosed interaction is skipped.

When to use `opt`:

An `opt` fragment is useful for optional notifications, logging under specific conditions, optional discount application, or additional processing controlled by a feature flag.

Difference between `alt` and `opt`:

`alt`	`opt`
Represents alternative execution paths	Represents an optional execution path
Similar to `if / else`	Similar to `if` without `else`
Contains multiple alternatives	Contains one optional interaction block
Selects an applicable alternative	Executes the block only when its guard is true

7.3 Repeated Execution: `loop`

A `loop` fragment represents interactions that execute repeatedly.

Consider a college application that sends reminders to every student enrolled in a course.

```
for (Student student : enrolledStudents) {
    notificationService.sendReminder(student);
}
```

Here, the same method is called for each student in the collection.

Rather than drawing the same interaction repeatedly, a sequence diagram can use a `loop` fragment.

```
loop [For each enrolled student]

    ReminderService
        → NotificationService:
          sendReminder(student)
```

The loop condition is written inside the frame.

The messages enclosed within the fragment are repeated according to that condition.

A UML loop may also specify minimum and maximum iteration counts when needed.

When to use `loop`:

Loops are commonly used for sending bulk notifications, processing collections, performing retry operations, and iterating over records.

---

8. Complete Example: Student Course Enrollment

The following interaction summarizes how the enrollment process can be represented using a sequence diagram.

Participants

- `Student`
- `CollegePortal`
- `EnrollmentController`
- `EnrollmentService`
- `StudentRepository`
- `CourseRepository`
- `EnrollmentRepository`
- `NotificationPublisher`

Enrollment Flow

Step 1: Enrollment request

The student selects a course and clicks Enroll.

The college portal sends the request to `EnrollmentController`.

Step 2: Service invocation

The controller calls:

```
enrollmentService.enroll(studentId, courseId);
```

Step 3: Student validation

`EnrollmentService` retrieves the student using `StudentRepository`.

The service verifies that the student exists and has an active account.

Step 4: Course validation

The service retrieves the selected course through `CourseRepository`.

It checks whether the course exists and has available capacity.

Step 5: Duplicate enrollment check

The service uses `EnrollmentRepository` to determine whether the student is already enrolled in the course.

Step 6: Enrollment creation

If all validations succeed, the service creates an `Enrollment` object and saves it through `EnrollmentRepository`.

Step 7: Optional notification

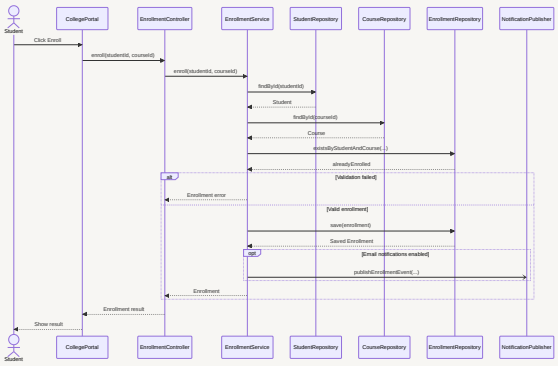
If email notifications are enabled, the application publishes an enrollment notification event.

Step 8: Response

The service returns the enrollment result to the controller, which sends the confirmation back to the college portal.

If a validation fails, the application returns an appropriate error instead.

Sequence Diagram — Mermaid Representation



This is a simplified logical view of the operation. In a production implementation, the exact validation sequence and handling of concurrent enrollment requests may require additional considerations.

---

9. Important Points for LLD Interviews

Sequence diagrams are particularly useful when explaining the internal execution of a feature.

They help communicate how responsibilities are distributed among objects and how those objects cooperate during an operation.

When creating sequence diagrams for LLD interviews:

1. **Start with one use case.** A sequence diagram should describe a particular interaction, such as enrolling in a course or placing an order.
2. **Identify the participants.** Include the objects or components directly involved in the operation.
3. **Show interactions in execution order.** Messages should follow the actual sequence of method calls and responses.
4. **Include important conditions.** Use `alt` or `opt` when conditional behavior affects the interaction.
5. **Use `loop` for repeated interactions.** Avoid manually duplicating the same sequence of messages.
6. **Distinguish synchronous and asynchronous operations.** The diagram should communicate whether an interaction requires waiting for completion.

7. **Avoid unnecessary implementation details.** Include the interactions needed to understand the design rather than every internal helper method.

A good sequence diagram should make it possible to understand how a particular feature works without reading the entire source code.

10. Quick Revision: Sequence Diagram Notations

Element	Purpose
Actor	Represents an external role interacting with the system
Participant	Represents an object, component, or system involved in the interaction
Lifeline	Represents a participant over the course of the interaction
Synchronous message	Represents a call that waits for completion
Asynchronous message	Represents a message that does not require the sender to wait for completion
Return message	Represents a reply or returned result
Activation bar	Represents a participant executing behavior
Self-call	Represents an object calling one of its own methods
Create message	Represents the creation of an object
Destruction marker (`X`)	Indicates the end of an object's lifetime
`alt`	Represents alternative execution paths
`opt`	Represents optional execution
`loop`	Represents repeated execution

Conclusion

Class diagrams and sequence diagrams serve different purposes in Low-Level Design.

A **class diagram** shows the structural relationships between classes, including their attributes, methods, and associations.

A **sequence diagram** shows how objects interact during a particular operation, including the order of method calls, responses, and conditional behavior.

Together, they provide a clearer understanding of both the structure and execution of an object-oriented system.

In machine coding and LLD interviews, sequence diagrams are especially valuable for explaining workflows that involve multiple collaborating objects or services.